Projet L3 : Plateforme Multi-Paradigmes de Gestion Intelligente de Tâches

1. Objectif général

Développer une application complète de gestion de tâches (type Trello simplifié) où **chaque fonctionnalité clé est implémentée avec un paradigme de programmation différent**. L'objectif est de comparer concrètement les paradigmes : **impératif, orienté objet, fonctionnel, logique et événementiel**.

2. Description globale du projet

L'application permet :

- Créer, modifier, supprimer des tâches
- Classer les tâches (priorité, catégorie)
- Planifier (dates, dépendances)
- Recommander automatiquement des actions
- Interface interactive (CLI ou GUI simple)

3. *Partie Conception (obligatoire avant implémentation)*

Avant de coder, les étudiants doivent produire une **conception complète** du système :

1.1 Analyse des besoins

- Identifier les fonctionnalités :
  - gestion des tâches
  - priorisation
  - dépendances
  - recommandations intelligentes
- Identifier les acteurs :
  - utilisateur simple
  - administrateur (optionnel)

1.2 Modélisation UML

- Diagramme de classes :
  - Task, User, Project, Category
- Diagramme de cas d'utilisation :
  - ajouter tâche
  - marquer comme terminée
  - consulter recommandations

4. Répartition par paradigme

A. Paradigme Impératif (C ou Python procédural)

Module : Gestion basique des tâches

- CRUD (Create, Read, Update, Delete)
- Stockage (tableaux, fichiers)
- Tri (par date, priorité)

Objectif pédagogique :

- Maîtriser structures de contrôle, boucles, tableaux, fichiers

B. Programmation Orientée Objet (Java / Python)

Module : Modélisation du système

- Classes :
  - **Task** (id, title, status, due_date, user_id, priority project_id)
  - **User** (id, name, email)
  - **Project** (id, name, start_date, end_date)
- Héritage :
  - SimpleTask et ComplexTask qui héritent de Task
- Design patterns :

    - MVC

C. Programmation Fonctionnelle (Python / JS)

Module : Analyse et filtrage des tâches

- Fonctions pures :
    - filtrer les tâches urgentes
    - calculer les statistiques
- Calculez le nombre de tâches terminées
- Transformez les listes de tâches
- Utilisation de :
    - `map`, `filter`, `reduce`

D. Programmation Logique (Prolog)

Module : Moteur de règles / recommandation

- Implémenter les règles suivantes :
    - "Si deadline proche → priorité haute"
    - "Si tâche dépend d'une autre non finie → bloquée"
- Requêtes :
    - tâches critiques ?
    - ordre optimal d'exécution ?

Objectif :

- Raisonnement déclaratif
- Inférence logique

E. Programmation Événementielle (JavaScript / Python GUI)

Module : Interface utilisateur

- Interface CLI interactive ou GUI (Tkinter / JS)
- Gestion des événements :
    - clic bouton
    - ajout tâche
    - notification deadline

Objectif :

- Programmation basée sur événements
- Callbacks / listeners

5. Livrables attendus

- Code source structuré (par paradigme)
- Rapport :
    - description des paradigmes
    - comparaison (avantages/inconvénients)